Task 3 - Parallel Bubble Sort

p_bubble_1.c

The array is split into one section per thread, and each thread bubble sorts its own section. At the end of each pass, a thread also compares its last element with the first element of the next section so values can move between sections. A barrier makes the threads wait for each other before starting the next pass.

Advantages: It is simple and splits the work evenly. It was faster than sequential with 2 threads (7.8 s vs 10.0 s).

Limitations: The boundary swap is a race condition because the next thread could be using that same element at the same time. It also always runs all 100,000 passes, never shrinks the range like the sequential version does, and calls the barrier every pass. Because of this it got slower with 3 and 4 threads, and the 1 thread version was slower than sequential.

p_bubble_2.c

This version uses odd-even (brick) sort. Even phases compare pairs (0,1), (2,3), ... and odd phases compare pairs (1,2), (3,4), ... The array is split between the threads with a barrier after each phase.

Advantages: The pairs in a phase never overlap, so threads never touch the same element at the same time and no mutex is needed. It was faster than sequential at every thread count.

Limitations: It is still O(n^2) and needs a barrier after all 100,000 phases. Each phase does very little work, so the barrier overhead adds up and it got slower with 3 and 4 threads.

Compared to sequential, both versions help a little, but they are still O(n^2). Merge sort sorted the same array in about 10 ms compared to about 10 s for bubble sort, so the algorithm matters more than the number of threads.

Task 4 - Benchmark Results

N = 1,000,000,000 for integration and SIZE = 100,000 for sorting.

Sequential times are the average of all runs.

Program	Sequential	1 Thread	2 Threads	3 Threads	4 Threads
Num_Integrate	2.661 s	2.732 s	1.393 s	.958 s	.769 s
Merge_approach_1	9.601 ms	10.252 ms	4.696 ms	3.694 ms	3.488 ms
Merge_approach_2	9.601 ms	8.782 ms	6.409 ms	5.298 ms	3.015 ms
Merge_approach_3	9.601 ms	10.226 ms	5.359 ms	4.579 ms	3.419 ms
Bubble_1	10.002 s	10.894 s	7.770 s	7.954 s	8.539 s
Bubble_2	10.083 s	5.648 s	5.328 s	6.087 s	7.108 s

Task 5 - Discussion Post

Observations

Numerical integration had the best speedup, about 3.5x with 4 threads. Each thread works on its own range with a local sum and only locks the mutex once, so the threads barely wait on each other.

All three merge sorts were around 3x faster with 4 threads. With 1

thread they were about the same as sequential because of the overhead of creating threads. The recursive approach was unexpected at first. It took around 20 ms with 4 threads because every recursive call locked the mutex. After I only allowed threads to be created in the top levels of the recursion, it dropped to 3 ms.

The bubble sorts were fastest with 2 threads and got slower after that. Both use a barrier on every pass, so adding threads added more waiting. This showed me that more threads is not always faster.

Key Considerations

- Splitting the work evenly, including leftover elements
- Keeping shared data to a minimum, like using a local sum
- Picking the right tool: a mutex for shared values, barriers for the bubble sort passes, and condition variables for the thread pool
- Limiting the number of threads in the recursive merge sort
- Giving every version the same input so the timing is fair

Benefits and Challenges of pthreads

The main benefit is that threads share memory and pthreads gives full control over creating and syncing them, which can give a big speedup on work that splits up well. The challenges are race conditions that are easy to miss, synchronization overhead that can make the program slower, and bugs that are hard to track down since the threads run in a different order every time.